NeuroGolf 2026

Strategy & Approach Document

The 2026 NeuroGolf Championship · Kaggle · IJCAI-ECAI 2026

Executive Brief

This document lays out our strategy for the 2026 NeuroGolf Championship: the competition structure, our hybrid DSL-plus-learned-network approach, the architecture of our solver pipeline, the baseline we have already established (16 of 400 tasks solved for an expected score of ~300), and the concrete iteration roadmap we will execute over the remaining 12 days before the submission deadline. The goal is not just to participate but to place in the top tier of the leaderboard and contend for the $50,000 prize pool.

Key Numbers

Metric	Value	Note
Total tasks	400	ARC-AGI public training v1
Baseline solved	16 / 400	4.0% coverage
Baseline score	≈ 300	out of a theoretical 10,000
Days remaining	12	until July 15, 2026 23:59 UTC
Total prize pool	$50,000	First prize $12,000

1. Competition Overview

The 2026 NeuroGolf Championship is a research-prediction competition hosted on Kaggle by the Neurosynthetic Research Institute, affiliated with the IJCAI-ECAI 2026 conference in Bremen. Unlike conventional ML competitions that reward predictive accuracy, NeuroGolf inverts the objective: participants must design the smallest possible neural networks that correctly solve ARC-AGI grid-transformation tasks. Every submitted network is scored on a combination of parameter count and memory footprint, creating a pressure to express each transformation as tersely as possible. The competition runs from April 15, 2026 to July 15, 2026, with a total prize pool of $50,000 and an additional $10,000 "Longest Leader" award for the team holding first place for the longest cumulative time during the competition window.

1.1 Task Structure

Each of the 400 tasks comes from François Chollet's ARC-AGI public training set (v1). Every task is presented as a series of small grid transformations: typically five input/output training pairs and one held-out test pair. Grids use ten colors (0-9) and range in size from 1×2 to 30×30 cells. Our submitted network must reproduce the transformation for any input that exemplifies the task — not just the public examples. The hosts hold a small private benchmark suite specifically to penalize overfitting, so solutions must generalize beyond memorization.

Submissions are packaged as submission.zip containing at most one ONNX network per task, named task001.onnx through task400.onnx. Each network must accept a statically-shaped (1, 10, 30, 30) float32 input tensor — a one-hot encoding of the 30×30 padded grid — and produce a tensor of the same shape whose argmax-over-channels yields the output grid. Files are limited to 1.44 MB each. The ONNX operators Loop, Scan, NonZero, Unique, Script, and Function are banned, ruling out most iterative and set-based idioms.

1.2 Scoring Formula

For any task where our network is functionally correct across all public and private test pairs, we earn:

```
score = max(1, 25 − ln(cost)) where cost = #parameters + #bytes
```

The natural logarithm makes the curve steep at small costs and flat at large ones. A network with cost 10 scores 22.7; cost 100 scores 20.4; cost 1,000 scores 18.1; cost 10,000 scores 15.8. The implication is that the first 10× reduction in cost is worth about 2.3 points, but going from 10,000 to 1,000 is worth the same 2.3 points. Every order of magnitude matters equally, which favors aggressive minimization. Even a network with cost 1,000,000 still earns the floor of 1 point, so shipping any functionally-correct network is strictly better than skipping a task.

1.3 Why This Competition Is Different

Standard deep-learning competitions reward scaling: bigger models, more data, longer training. NeuroGolf rewards compression: the smallest model that solves a task wins. This

makes it a neural program synthesis problem rather than a supervised-learning problem. The right tool is not PyTorch training loops but a domain-specific language (DSL) of ARC primitives that can be assembled into tiny ONNX graphs. Where learned networks are unavoidable (because a task's rule is too irregular to capture by hand), we still want micro-networks — a few hundred parameters at most, trained with strong L1 / size penalties and distilled into deterministic form.

1.4 Competitive Landscape

As of July 3, 2026 the competition has 8,153 entrants, 3,156 active participants, 2,799 teams, and 378,207 total submissions — an average of ~120 submissions per team. This is a highly active competition. The "Longest Leader" prize of $10,000 rewards holding first place for the longest cumulative time between May 6 and July 15, which means early leaderboard presence matters as much as the final rank. Our strategy therefore emphasizes shipping a working submission early (which we already have), then iterating aggressively on coverage and cost to climb.

2. Our Approach: Hybrid DSL + Learned Micro-Nets

We chose a hybrid strategy combining hand-crafted DSL primitives with learned micro-networks. The hand-crafted DSL covers ARC task families that admit a clean algebraic description (color maps, geometric transforms, tiling, Kronecker products, shifts, neighbor-count rules). Learned micro-networks — small convolutional stacks trained per-task with size penalties — cover the long tail of irregular tasks that do not fit any DSL primitive. The two layers are not mutually exclusive: a typical task may be partially solved by a DSL primitive (e.g., a color map) with a learned residual on top.

2.1 I/O Convention

We reverse-engineered the I/O convention from the competition's single example (a 3×3 single-layer convolutional network with 900 parameters). The validator appears to:

- Pad the input grid to 30×30 with zeros (color 0)
- One-hot encode to (1, 10, 30, 30) float32 — one channel per color 0-9
- Run our ONNX network on this tensor
- Take argmax over the channel dimension to recover a 30×30 grid
- Crop the result to the expected output dimensions (which the validator knows from the test pair)

This convention has one important consequence: the network's output is always 30×30, and only the top-left (out_H, out_W) region matters. This lets us implement scaling, tiling, and concat operations uniformly without needing variable-shape outputs.

2.2 DSL Primitives

Each DSL primitive is a Python function that returns an onnx.ModelProto. Primitives are deliberately small (most are under 1 KB serialized) and composable via a chain() operator.

The current catalog includes:

Primitive	Params	Typical Score	Covers
identity	0	20.1	Pass-through tasks
color_map	100	18.4	Per-color substitution (1×1 conv)
geom_transform (flip/rot/transpose)	0	20.0	Geometric transforms
scale_up (k×)	0	19.2	Nearest-neighbor upscaling
shift (dh, dw)	0	19.5	Translation with zero padding
tile (k×k)	0	19.5	Pattern repetition
kronecker (k×k)	0	18.0	Conditional tiling (007bbfb7 family)
concat_repeat	0	19.5	Extend grid by repeating rows/cols
slice_colormap	100	18.0	Slice a sub-region + color map
cellular_automaton (1 rule)	120	17.7	Neighbor-count threshold rules
multi_rule_ca	200+	16.5	Multiple neighbor-color → output-color rules
constant_grid	10·H·W	6-10	Last-resort constant output

2.3 Solver Dispatch

For each task, we run every solver in cost order (cheap primitives first, expensive ones last) and pick the smallest eligible network — where "eligible" means the network passes structural checks (no banned ops, static shapes, under 1.44 MB) and functional checks (correctly transforms every public pair in the task). Each solver is wrapped in a uniform Solver interface that returns either a SolverResult or None. The dispatcher's runtime is dominated by ONNX inference (~50 ms per solver per task); the full 400-task sweep completes in under a minute on a single CPU core.

2.4 Local Validator

Because we cannot see the private benchmark, we built a local validator that mirrors the competition's structural checks (file size, op blacklist, static shapes, onnx.checker) and functional checks (run on all train + test pairs and compare to expected output). This validator runs after every solver attempt and gates which networks make it into submission.zip. The validator cannot catch overfitting to the public pairs — only the private benchmark will — so we deliberately prefer solvers that express general rules ("swap colors 2 and 5") over solvers that memorize specific layouts.

3. Codebase Architecture

The solution codebase lives at /home/z/my-project/neurogolf/ and is structured as a Python package with clear separation between data, DSL, solvers, validation, and submission packing. All scripts are persisted under /home/z/my-project/scripts/ so they can be edited and re-run as we iterate (per the Script Persistence Rule).

3.1 Module Map

Module	Role
constants.py	I/O shape, banned ops, scoring formula
arc_data.py	Load ARC-AGI JSON, one-hot encode/decode, task signatures
dsl.py	ONNX-emitting primitives: identity, color_map, conv, chain, argmax
validator.py	Structural + functional validation, cost & score computation
solvers/base.py	Solver abstract class + dispatcher (run_solvers)
solvers/simple.py	Identity, color_map, replace_color, constant
solvers/transforms.py	Flip, rotate, transpose, color_map + transform
solvers/advanced.py	Scale, crop, shift, tile, kronecker, concat, slice
solvers/patterns.py	Mirror concat, palette, exhaustive color map
solvers/cellular.py	Single-rule CA and multi-rule CA (neighbor-fill)
build_submission.py	End-to-end pipeline: tasks → solvers → submission.zip

3.2 Data Flow

The end-to-end pipeline is:

```
ARC-AGI JSON → arc_data.load_task → solver dispatch → ONNX model → validator → submission.zip
```

Each task is loaded once, all solvers are tried in order, and the smallest eligible ONNX model is written directly into the zip under the canonical name taskNNN.onnx. Tasks with no eligible solver are simply omitted from the zip — they contribute zero to the score but do not penalize us. The current submission.zip is 5.6 KB and contains 16 ONNX files.

3.3 Reproducibility & Iteration

Every solver is deterministic (no randomness, no learning). Running the pipeline twice produces byte-identical output. This is critical for debugging: when a solver regresses, we can bisect by running on individual tasks. The full 400-task sweep completes in ~50 seconds on the CPU sandbox, so we can iterate many times per session. A JSON dump of every per-task result (solver name, cost, score, eligibility) is written to data/submission_results.json for offline analysis.

4. Baseline Results

Our first-pass baseline — built in this single session — solves 16 of 400 tasks for an expected score of approximately 300. The theoretical maximum (if every task scored the ceiling of 25) would be 10,000, so we are at roughly 3% of the ceiling. The realistic competitive target, based on the score distribution in similar ARC-AGI competitions, is in the 2,000-3,500 range — meaning we have substantial room to climb.

4.1 Solver Breakdown

Solver	Tasks Solved	Avg Score	Notes
color_map	4	18.4	1×1 conv per-color substitution
crop_top_left	3	20.1	Identity network + validator crop
cellular_automaton	3	17.7	Single (X,Y,Z,threshold) rules
geom_transform	2	20.0	Flip / transpose
scale_up	2	19.2	Nearest-neighbor 2× scaling
kronecker	1	18.0	Conditional tiling
multi_rule_ca	1	16.5	Multiple neighbor-color rules
Total	16	18.8 avg	—

4.2 Failing-Task Categorization

We categorized the 384 currently-unsolved tasks by structural signature to identify where to focus iteration effort:

Category	Count	% of Fails	Tractability
Same-size, <30% cells changed	177	46%	High — likely CA / pattern rules
Same-size, complex change	79	21%	Medium — needs multi-step reasoning
Different I/O sizes	132	34%	Medium — crop/tile/scale/concat families
1-D outputs (row/column)	14	4%	Hard — count/extraction logic

The largest opportunity is the 177 same-size low-diff tasks: these are very likely cellular-automaton or pattern-based recoloring rules that our existing CA solver should be able to capture with extensions (e.g., 8-neighbor rules, multi-rule chaining, position-conditional rules). Capturing even 30% of these would more than double our coverage.

4.3 Score Distribution of Solved Tasks

Of the 16 solved tasks, scores range from 16.5 (multi-rule CA, the densest network) to 20.1 (identity and crop, the lightest). The average of 18.8 corresponds to a median cost of around 700 — squarely in the "a few hundred parameters" range. There is room to push individual scores higher by re-encoding primitives more tersely (e.g., using int8 instead of float32 for conv weights), but the bigger win is increasing coverage rather than optimizing per-task cost.

5. Iteration Roadmap (12 Days Remaining)

Our plan is organized into four phases, each with concrete deliverables and a target coverage / score. The phases overlap — we will keep refining earlier solvers even as we

add new ones — but each phase has a primary objective.

Phase 1 — Coverage Push (Days 1-3)

Goal: triple coverage from 16 → 50+ tasks, score ~1,000. The primary lever is extending the CA solver family to handle the 177 same-size low-diff tasks. Specific extensions: (a) 8-neighbor rules with arbitrary (X, Y, Z, threshold) combinations; (b) multi-rule chaining where each rule produces a candidate output and the final network combines them; (c) "count → color" rules where the output color is the neighbor count itself (task 10 family); (d) position-conditional color maps (task 10 column-based replacement). We will also add a fill-enclosed solver for flood-fill tasks (task 2 family) using iterated dilation.

Phase 2 — Geometric & Scaling Families (Days 4-6)

Goal: coverage 50 → 100+ tasks, score ~2,000. Target the 132 different-I/O-size tasks. Add: (a) symmetric-completion solver (concat input + mirror); (b) scale-then-color-map; (c) extract-then-color-map with marker-based sub-region detection; (d) draw-line-between-markers using Bresenham-equivalent convolutions; (e) multi-tile patterns where the output is the input repeated N×N times. Each new solver is benchmarked against the full 400-task set and the baseline is re-measured before committing.

Phase 3 — Learned Micro-Nets (Days 7-9)

Goal: coverage 100 → 200+ tasks, score ~3,500. For tasks that no DSL primitive solves, train per-task micro-networks: 1-3 layer conv stacks with 32-128 channels, trained on the (input, output) pairs with strong L1 / size penalty. Use straight-through estimators to handle the argmax non-differentiability. Distill the trained network into a sparse form (prune near-zero weights, quantize to int8) before exporting to ONNX. This phase requires CPU-intensive training; we will need to either rent cloud GPU or run training on the user's local machine.

Phase 4 — Cost Optimization & Final Push (Days 10-12)

Goal: maximize score per task via aggressive cost reduction. Re-encode all networks to use the smallest possible data types (int8 weights, float16 activations where safe). Fold identity operations, eliminate redundant constants, and minimize intermediate tensor names. For tasks where we already have a working network, search for alternative solvers that produce a smaller-equivalent network. Submit daily to climb the "Longest Leader" ranking, with a final submission on the morning of July 15.

5.1 Daily Cadence

Each day: (1) pick one solver family from the phase plan; (2) implement and unit-test it on hand-curated examples; (3) run the full 400-task baseline; (4) commit only if coverage or score improves; (5) submit to Kaggle; (6) log the result in worklog.md. This tight loop ensures we always have a working submission and can roll back easily if a change regresses.

6. Risks & Mitigations

6.1 Private Benchmark Overfitting

The single largest risk is that our networks, while correct on public pairs, fail the private benchmark because they encode task-specific patterns rather than general rules. For example, a color map that swaps 2↔5 because all training pairs happen to use those colors will fail if the private test uses 3↔7. Mitigation: prefer solvers that express structural rules ("swap the two most-common non-zero colors") over solvers that hard-code specific color indices. Where we must hard-code, document the assumption so we can revisit during the final review.

6.2 I/O Convention Misassumption

We inferred the (1, 10, 30, 30) one-hot I/O convention from the competition's single example. If the actual validator uses a different convention (e.g., a single-channel integer grid, or a different padding rule, or a separate output-dimensions tensor), all our networks will fail. Mitigation: ship the current submission early (which we have) to confirm at least some tasks score >0. If the first submission scores 0 across the board, we will revisit the convention.

6.3 Compute Constraints

We are running on a CPU-only sandbox, which limits our ability to train learned micro-nets (Phase 3). A single per-task training run that would take 30 seconds on GPU might take 30 minutes on CPU, making 200 tasks impractical. Mitigation: the user has indicated they have a Kaggle account and can run Kaggle Notebooks (free GPU/TPU, 9-hour limit). We will package the training script as a Kaggle Notebook so the user can launch it on free GPU time, then download the trained ONNX files for inclusion in our submission.

6.4 ONNX Operator Compatibility

We have already encountered ONNX operator pitfalls (Resize nearest_mode interpretation, OneHot input count, Mul on bool tensors). The competition's validator may use a different ONNX runtime version that handles these differently. Mitigation: keep the ONNX opset version pinned (we use opset 17), test with both onnxruntime 1.27 and an earlier version, and prefer well-supported ops (Conv, Mul, Add, Sub, Slice, Concat, Constant, Cast, Pad, Resize, Tile, ArgMax, OneHot, Transpose) over exotic ones.

6.5 Time Pressure

Twelve days is tight for a competition of this complexity. The realistic outcome is that we will not solve all 400 tasks. Mitigation: prioritize coverage breadth (one network per task, even if some are sub-optimal) over depth (perfecting individual tasks). A submission with 200 tasks at average score 15 (3,000 points) beats a submission with 50 tasks at average score 20 (1,000 points).

6.6 Risk Matrix Summary

Risk	Likelihood	Impact	Mitigation
Overfitting private set	Medium	High	Prefer structural rules over specific colors
I/O convention wrong	Low	Critical	Ship early to validate; document assumptions
Compute limits	High	Medium	Use Kaggle Notebooks for GPU training
ONNX op incompatibility	Medium	Medium	Pin opset 17, test on multiple runtimes
Time pressure	High	Medium	Prioritize coverage over perfection

7. Immediate Next Steps

Concretely, in the next session we will execute the following work items in priority order. Each item is sized to fit in a single focused work session, and each produces a measurable improvement to the baseline.

- Upload the current submission.zip to Kaggle and confirm that at least some tasks score > 0. This validates our I/O convention assumption before we invest further.
- Extend the CA solver with 8-neighbor rules and multi-rule chaining. Target: solve 20+ of the 177 same-size low-diff tasks.
- Add a symmetric-completion solver (concat input + flip). Target: solve 5-10 of the different-I/O-size tasks.
- Add a flood-fill solver using iterated 3×3 dilation. Target: solve 5-10 enclosed-region tasks.
- Profile and optimize the dispatcher runtime — currently 50s for 400 tasks is acceptable but will grow as we add solvers.
- Build a Kaggle Notebook template for the user to run learned-micro-net training on free GPU in Phase 3.

All progress is logged in /home/z/my-project/worklog.md using a shared multi-agent protocol so any future session can pick up where we left off. The submission file is regenerated from scratch each session by running python3 -m neurogolf.build_submission, so we are always shipping the best-known solution.